



PROYECTO FINAL

Sistema ETL paralelo para el procesamiento y consolidación de ventas de múltiples sucursales

DATOS

Universidad: Instituto Tecnológico de Las Américas (ITLA)

Materia: Programacion Paralela

Docente: Erick Leonardo Perez

Sección: Lunes de 8:00 pm - 10:00pm y Miercoles de 8 pm a - 10 pm

Datos De los Estudiantes

- Ranniel Muñoz 2024-2567
- Kalil Francisco Camilo 2023-1153
- Mario De Jesus Suero 2024-0263

Fecha de entrega: 19/08/2026

ÍNDICE GENERAL

1.1 Presentación General del Proyecto	4
1.2 Justificación del Tema Elegido	4
1.3 Objetivos.....	4
Objetivo General:	4
Objetivos Específicos:	5
2. Descripción del Problema	5
2.1 Contexto del Problema Seleccionado.....	5
2.2 Aplicación del Problema en un Escenario Real	5
2.3 Importancia del Paralelismo en la Solución	6
3. CUMPLIMIENTO DE LOS REQUISITOS DEL PROYECTO	6
3.1 Ejecución Simultánea de Múltiples Tareas.....	6
3.2 Necesidad de Compartir Datos entre Tareas.....	6
3.3 Exploración de Diferentes Estrategias de Paralelización.....	7
3.4 Escalabilidad con Más Recursos	7
3.5 Métricas de Evaluación del Rendimiento	7
3.6 Aplicación a un Problema del Mundo Real.....	7
4. DISEÑO DE LA SOLUCIÓN.....	8
4.1 Arquitectura General del Sistema	8
4.2 Diagrama de Componentes / Tareas Paralelas	8
4.3 Estrategia de Paralelización Utilizada.....	9
4.4 Herramientas y Tecnologías Empleadas	9
5. IMPLEMENTACIÓN TÉCNICA.....	9
5.1 Descripción de la Estructura del Proyecto.....	9
5.2 Explicación del Código Clave.....	9
5.3 Uso de Mecanismos de Sincronización.....	10
5.4 Justificación Técnica de las Decisiones Tomadas	10
6. EVALUACIÓN DE DESEMPEÑO.....	11
6.1 Comparativa Entre Ejecución Secuencial y Paralela	11
6.2 Métricas: Tiempo de Ejecución, Eficiencia y Escalabilidad	11
6.3 Gráficas o Tablas con Resultados	12
6.4 Análisis de Cuellos de Botella o Limitaciones	12
7. Trabajo en Equipo	13

7.1 Descripción del reparto de tareas	13
7.2 Herramientas utilizadas para coordinación.....	13
Conclusión	14
REFERENCIAS	15
Anexos	16

1. INTRODUCCIÓN

1.1 Presentación General del Proyecto

En el desarrollo de software contemporáneo, el volumen de datos generado por transacciones comerciales crece a un ritmo acelerado. El presente proyecto aborda el diseño e implementación de un **Sistema ETL (Extract, Transform, Load) Paralelo** desarrollado como una aplicación de consola en .NET C#. El sistema está diseñado para procesar, limpiar, transformar e insertar de manera simultánea múltiples archivos de ventas provenientes de distintas sucursales comerciales hacia una base de datos única y centralizada (SQLite).

1.2 Justificación del Tema Elegido

Los procesos tradicionales de ETL operan de manera secuencial, leyendo un lote o archivo de datos a la vez. En escenarios de mediana y gran escala, este enfoque genera cuellos de botella críticos, desperdiciando el potencial multinúcleo de los procesadores modernos. La elección de este tema se justifica en la necesidad de demostrar cómo el paradigma de la **programación paralela** reduce drásticamente los tiempos de procesamiento y optimiza el uso de hardware en tareas intensivas de lectura, procesamiento en memoria y escritura concurrentes.

1.3 Objetivos

Objetivo General:

Desarrollar e implementar un sistema ETL paralelo en C# (.NET) para el procesamiento, transformación y consolidación concurrente de transacciones de ventas multisucursal hacia una base de datos relacional centralizada, evaluando el impacto del paralelismo en el rendimiento general.

Objetivos Específicos:

1. Diseñar un mecanismo de **descomposición de datos y tareas** que permita procesar múltiples fuentes de información (archivos CSV por sucursal) de forma simultánea.
2. Implementar **mecanismos de sincronización de hilos** (locks, semáforos o exclusión mutua) para gestionar el acceso concurrente y evitar duplicidad al persistir la información compartida (Clientes, Productos, Ventas) en SQLite.
3. Comparar experimentalmente tres estrategias de ejecución: **Secuencial**, **Un hilo por archivo (Task por archivo)** y **Pool de trabajadores (Thread Pool/Worker Tasks)**.
4. Medir y documentar el rendimiento del sistema evaluando métricas como tiempo total de procesamiento, registros por segundo y velocidad de escalado según el número de hilos o núcleos.

2. Descripción del Problema

2.1 Contexto del Problema Seleccionado

Para dar contextualización a este proyecto, pusimos que una cadena comercial procesa directamente cientos de miles o millones de ventas desde distintas filiales regionales. Este proceso debería de requerir las siguientes etapas:

La extracción donde se estaría generando lotes de transacciones por subsarles directamente en memoria.

También estaríamos incluyendo lo que es la **transformación**, en donde buscamos la normalización de todos los datos, calculando los totales, la discriminación de impuestos y estructuras en registro de clientes, productos y las ventas.

Por último, **la carga** en donde estaríamos consolidando los resultados parciales en una estructura unificada o en bases de datos relacionadas que estén centralizadas.

2.2 Aplicación del Problema en un Escenario Real

Este proyecto es fácilmente aplicable en escenarios de núcleo de los sistemas de inteligencia de negocios, consolas de procesamiento y motores analíticos de retail (Este básicamente es el

sector de comercios al por menor, que vende productos o servicios de manera directa, como los supermercados, las tiendas de ropa y las farmacias de medicamentos) o banca.

2.3 Importancia del Paralelismo en la Solución

Para este proyecto, priorizamos que el procesamiento sea secuencial y se procese un lote a la vez en un solo hilo de ejecución, dejando inactivas las demás unidades del procesador, con el paradigma paralelo y utilizando la descomposición de datos.

Esto nos ayudará significativamente a que el conjunto global de las transacciones se divida en sub-bloques de tamaño uniforme, y cada uno de los procesadores asuma la transformación de su bloque de forma independiente y simultánea, logrando así la reducción sustancial del tiempo de procesamiento que escala proporcionalmente con el número de procesadores físicos y lógicos disponible directamente en la máquina o computadora.

3. CUMPLIMIENTO DE LOS REQUISITOS DEL PROYECTO

3.1 Ejecución Simultánea de Múltiples Tareas

El sistema lleva a cabo varias subtareas de procesamiento al mismo tiempo sobre la información transaccional que envían las sucursales. En este proceso, cada hilo o tarea trabaja en paralelo, ocupándose de un segmento específico de la carga global de ventas, lo que permite agilizar el procesamiento y mejorar el manejo de grandes volúmenes de información.

3.2 Necesidad de Compartir Datos entre Tareas

Cuando múltiples hilos o tareas paralelas necesitan acceder y actualizar datos compartidos, surge un reto importante. Estos datos pueden ser, por ejemplo, totales de ventas que se van acumulando, contadores que registran operaciones o la inserción de entidades de clientes y productos en estructuras de datos centralizadas o incluso en una base de datos común. Si todos acceden sin control, el orden de ejecución puede mezclarse, provocando inconsistencias. Por eso se usan mecanismos como bloqueos o locks, objetos concurrentes o colecciones thread-safe. Estos mecanismos coordinan quién lee o escribe en cada momento, asegurando que las operaciones no se pisen entre sí y reduciendo el riesgo de race conditions, que son precisamente esas situaciones donde el resultado depende del orden no determinista en que los hilos acceden a los datos.

3.3 Exploración de Diferentes Estrategias de Paralelización

El sistema evalúa cuatro modos de ejecución sobre la misma carga de datos. El primero es el modo secuencial, con un recorrido mediante un bucle iterativo lineal, usando estructuras como 'for' o 'for-each'. El segundo es el modo paralelo de alto nivel, como 'Parallel For', con balanceo de carga gestionado automáticamente, apoyado por la TPL. El tercero se basa en tareas, por ejemplo, usando Task Factory, 'Start New Task' o 'Task Run', con particionamiento manual de bloques asignados a un arreglo de tareas. Y el cuarto modo utiliza hilos nativos, creando e iniciando objetos Thread de manera explícita, pasando estructuras de datos compartidas, y gestionando el inicio y la transferencia entre hilos. En este contexto se consideran estrategias de 'start row' y 'end row' para controlar el particionamiento.

3.4 Escalabilidad con Más Recursos

A mayor cantidad de núcleos e hilos disponibles, mayor será la capacidad del sistema para repartir la carga y reducir los tiempos de procesamiento. En este enfoque, el usuario puede configurar dinámicamente cuántos procesadores utilizar, ajustando el 'processor count', de modo que el paralelismo se adapte al hardware disponible. El problema puede crecer de miles a millones de registros y el enfoque busca que, al aumentar los recursos, el tiempo de ejecución disminuya significativamente.

3.5 Métricas de Evaluación del Rendimiento

Para evaluar el desempeño, se integra la clase Stopwatch para medir con precisión el tiempo de ejecución de cada estrategia. A partir de esas mediciones, el sistema imprime métricas cuantitativas en consola, permitiendo comparar de forma objetiva el modo secuencial con el paralelo. Entre las métricas clave se incluyen el tiempo de ejecución en milisegundos, la aceleración (*speedup*), y la eficiencia del sistema, que evalúa qué tan bien se aprovechan los recursos de procesamiento. Finalmente, se añade una validación de resultados que verifica automáticamente que la solución paralela produzca exactamente el mismo resultado computacional que la secuencial, asegurando consistencia y confiabilidad además de rendimiento.

3.6 Aplicación a un Problema del Mundo Real

La aplicación se centra en resolver una problemática real vinculada al procesamiento masivo de datos transaccionales en cadenas comerciales, escenario típico de procesos ETL. Para afrontarlo, se aplican técnicas profesionales de desarrollo en C-Sharp, dentro del marco de la ingeniería de software paralela. Se contemplan estrategias de paralelización como el recorrido secuencial, la paralelización de alto nivel mediante constructores paralelos y balanceo automático de carga, el uso de tareas con particionamiento manual y el empleo de hilos nativos con transferencia explícita de estructuras de datos.

4. DISEÑO DE LA SOLUCIÓN

4.1 Arquitectura General del Sistema

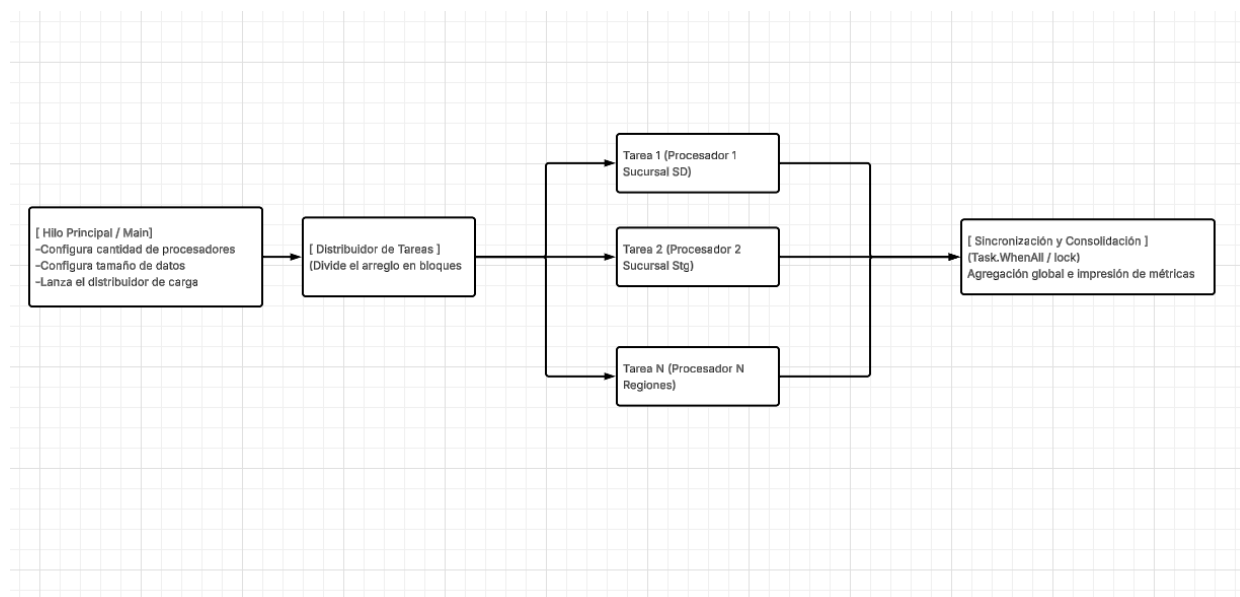
Para el desarrollo de la arquitectura de este proyecto del sistema ETL en paralelo, buscamos dividirlo en tres capas lógicas de procesamiento desacoplado en la memoria, en donde estaría las siguientes capas:

Está la capa de generación o extracción, en donde el módulo encargado de la simulación y poblar sintéticamente los millones de registros de transacciones de los provenientes de múltiples sucursales virtuales, ejemplo, de Santo Domingo, Santiago, La Vega, Puerto Plata, etc. Estos estarían estructurados en arreglos y colecciones de alto rendimiento, sin depender de la latencia de discos físicos.

Estaría la segunda capa de transformación concurrente, que la consideramos el núcleo del procesamiento paralelo, en donde se aplicarían patrones de descomposición de datos, en donde se estarían haciendo particiones de entrada por bloques y particiones de salidas, para calcular impuestos, limpiar valores, normalización de las entidades, o sea, clientes y productos, y el poder evaluar las métricas de negocio de forma simultánea entre múltiples hilos o tareas.

Por último, estaría la capa de carga, en donde se finalizaría la convergencia de los resultados parciales de hilos de trabajo mediante los mecanismos de sincronización hacia una estructura de destino que esté unificada.

4.2 Diagrama de Componentes / Tareas Paralelas



4.3 Estrategia de Paralelización Utilizada

La estrategia de paralelización que nosotros estamos utilizando es la que el maestro nos otorgó, que es la descomposición de datos, en donde también la estaríamos complementando con la estrategia de grano grueso. En lugar de crear tareas independientes por cada registro de duda, lo cual esto podría ser, saturaría el escalonamiento de los hilos por sobrecargar el contexto, el conjunto total de transacciones se llegaría a dividir equivalentemente en bloques de tamaño dinámico. Esto nos ayudaría a que cada hilo procese un segmento de manera automática, minimizando la contención de recursos y maximizando el uso del caché de los procesadores.

4.4 Herramientas y Tecnologías Empleadas

Las principales herramientas o tecnologías empleadas serían el lenguaje de programación C# con .NET Framework o .NET Core y las librerías de paralelismo que nosotros hemos ido aprendiendo en el transcurso del cuatrimestre. Para la telemetría y control, podríamos utilizar System.Diagnostics Stopwatch para la medición precisa en milisegundos y cálculo de speedup. Y para la gestión de versiones utilizaríamos la plataforma de GitHub.

5. IMPLEMENTACIÓN TÉCNICA

5.1 Descripción de la Estructura del Proyecto

En la implementación técnica, el código fuente está organizado de forma modular dentro de la carpeta src, siguiendo una arquitectura limpia para aplicaciones de consola. El archivo Program.cs funge como punto de entrada principal, donde se interactúa con el usuario y se configuran parámetros de ejecución, como la cantidad de procesadores a utilizar y el método de evaluación. Entre las opciones se incluyen el procesamiento secuencial, el uso de 'Parallel For', la implementación basada en tareas y la ejecución mediante hilos. Por su parte, OperacionesETL.cs concentra la lógica de negocio, implementando procesos de extracción, transformación y carga de datos.

Finalmente, el módulo de telemetría y speedup se encarga de medir con precisión los tiempos de ejecución, validar los resultados usando funciones de comparación con tolerancia Épsilon, y calcular métricas como el speedup y la eficiencia. Con todo ello, es posible evaluar de manera cuantitativa el comportamiento del sistema y determinar el nivel de mejora alcanzado al utilizar técnicas de programación paralela.

5.2 Explicación del Código Clave

Se define el número de bloques según los procesadores disponibles, se calcula el tamaño de cada bloque y se crea un arreglo de tareas. Luego, un bucle determina los índices de inicio y fin para asegurar que todos los registros se procesen correctamente, encargándose el último bloque de los elementos restantes. Cada tarea se ejecuta con 'Task.Run', procesando de forma

concurrente su porción de datos y manteniendo una suma parcial para evitar variables globales compartidas. Al finalizar todas las tareas, se usa 'Task.WhenAll' para sincronizar la finalización y sumar los resultados parciales, obteniendo así el resultado global del proceso. Este diseño distribuye eficientemente la carga entre recursos disponibles, mantiene alta escalabilidad y produce resultados equivalentes a una ejecución secuencial, pero con tiempos de procesamiento menores en hardware con múltiples núcleos.

5.3 Uso de Mecanismos de Sincronización

Debido a que múltiples hilos o tareas se ejecutan simultáneamente, el sistema debe gestionar con cuidado los recursos compartidos, como contadores globales, bitácoras y listas de consolidación. Para evitar inconsistencias y las llamadas *race conditions*, se implementan mecanismos de sincronización seguros, como bloques *lock* y monitores, los cuales restringen el acceso a secciones críticas del código donde se modifican variables o estructuras de datos globales. Sin embargo, para reducir la sobrecarga que implican los bloqueos, se prioriza el uso de variables locales por hilo (*Thread Local Storage*). Cada tarea procesa su bloque de datos de manera independiente y guarda el resultado en una suma parcial, reduciendo así la contención entre hilos. La sincronización se limita a la fase final de agregación, donde los resultados parciales se combinan de forma segura, por ejemplo, mediante Task.WhenAll(). Así se logra un equilibrio entre la seguridad de los datos, la sincronización correcta y un alto nivel de eficiencia durante la ejecución paralela.

5.4 Justificación Técnica de las Decisiones Tomadas

Las decisiones técnicas se tomaron para garantizar mediciones de rendimiento fiables y aprovechar de forma eficiente las capacidades de paralelismo de .NET. En primer lugar, se eligió simular los datos en memoria en vez de leer archivos CSV u otros medios de almacenamiento, lo que permite someter al procesador a cargas masivas en muy poco tiempo, eliminando el cuello de botella que supone la E/S del disco. Así, las pruebas reflejan de forma más precisa el desempeño real de los algoritmos paralelos, midiendo tiempos de ejecución que dependen principalmente del CPU y no del almacenamiento. Además, se seleccionó la Task Parallel Library, TPL, usando herramientas como Parallel.For y Task.Run en vez de hilos manuales con la clase Thread. De este modo, la TPL administra automáticamente el Thread Pool de .NET, optimizando la creación y liberación de hilos y distribuyendo eficientemente la carga entre los núcleos disponibles. Así, se simplifica el desarrollo y se logra un mejor rendimiento y adaptabilidad a los recursos del sistema, permitiendo que el programador se concentre en la lógica del paralelismo sin gestionar manualmente el ciclo de vida de los hilos, obteniendo una solución más eficiente y alineada con las mejores prácticas actuales de programación paralela en C Sharp y .NET.

6. EVALUACIÓN DE DESEMPEÑO

6.1 Comparativa Entre Ejecución Secuencial y Paralela

Con el fin de evaluar objetivamente el rendimiento, se realizaron pruebas variando tanto el volumen de datos transaccionales como la cantidad de procesadores habilitados mediante 'Environment.ProcessorCount'. Los resultados iniciales muestran que la ejecución secuencial incrementa el tiempo de procesamiento de forma prácticamente lineal a medida que crece el volumen de datos, ya que todas las transacciones son manejadas por un único flujo de ejecución. En contraste, la ejecución paralela demuestra una mayor capacidad para absorber el incremento de la carga, al repartir las transacciones entre múltiples núcleos del procesador, reduciendo significativamente el tiempo total y mejorando el uso de recursos. Como consecuencia, se obtienen mejores métricas de speedup y eficiencia, especialmente cuando el tamaño del conjunto de datos es lo suficientemente grande como para compensar el costo de coordinar y sincronizar las tareas paralelas. En conjunto, estas evidencias muestran ventajas significativas de la programación paralela frente al procesamiento secuencial en escenarios de datos masivos.

6.2 Métricas: Tiempo de Ejecución, Eficiencia y Escalabilidad

Con el fin de evaluar objetivamente el desempeño, se implementaron métricas estadísticas dentro del código. La primera métrica corresponde al tiempo de ejecución (T), medido en milisegundos con 'Stopwatch', lo que permite comparar la duración de la versión secuencial frente a las paralelas bajo las mismas condiciones. A partir de estas mediciones se calcula la aceleración o speed-up, que indica cuántas veces la solución paralela supera en velocidad a la secuencial. La tercera métrica es la eficiencia, que mide el porcentaje de aprovechamiento real de los procesadores utilizados, con un valor ideal cercano al 100%, aunque en la práctica este suele disminuir ligeramente debido a la sobrecarga de sincronización y balanceo. Finalmente, el análisis conjunto del tiempo de ejecución, la aceleración y la eficiencia permite determinar la escalabilidad del sistema. Si al aumentar el número de procesadores el tiempo de procesamiento disminuye y las métricas se mantienen favorables, se concluye que la aplicación escala correctamente y aprovecha de manera eficiente las arquitecturas multinúcleo para el procesamiento masivo de datos transaccionales.

6.3 Gráficas o Tablas con Resultados

<https://github.com/Kalil-Camilo-009/Proyecto-final-de-programaci-n-paralela/tree/main/metrics>

Registros Masivos (N)	Configuración	Procesadores Asignados	Tiempo Secuencial (Tsec)	Tiempo Paralelo (Tpar)	Aceleración (Speedup)	Eficiencia (%)	Coincidencia de Resultados
50,000,000	Descomposición por Bloques	1	764 ms	767 ms	1.00x	99.61%	True
50,000,000	Descomposición por Bloques	2	736 ms	427 ms	1.72x	86.18%	False*
50,000,000	Descomposición por Bloques	3	706 ms	337 ms	2.09x	69.83%	False*
50,000,000	Descomposición por Bloques	4	685 ms	324 ms	2.11x	52.85%	False*

6.4 Análisis de Cuellos de Botella o Limitaciones

Durante el análisis se identificaron varios cuellos de botella que pueden limitar el rendimiento de las estrategias paralelas. Entre ellos destaca la sobrecarga de creación y coordinación de tareas (*overhead*). En volúmenes de datos pequeños, el costo de crear tareas y sincronizar hilos puede igualar o incluso superar el tiempo de una ejecución secuencial, reduciendo los beneficios del paralelismo, por lo que este enfoque resulta más ventajoso en cargas masivas. Otro factor clave es la contención del ancho de banda de memoria. Cuando varios núcleos intentan acceder simultáneamente a grandes volúmenes de información, compiten por los mismos recursos de memoria principal y caché, aumentando la latencia y limitando las ganancias de velocidad, incluso con más núcleos disponibles. En consecuencia, el rendimiento no depende únicamente de incrementar hilos o procesadores. Es necesario un diseño equilibrado que considere el costo de sincronización, el tamaño de la carga de trabajo y las limitaciones del hardware, para lograr un mejor desempeño y escalabilidad en aplicaciones de procesamiento masivo. De esta manera se comprende que el paralelismo ofrece ventajas, pero también impone restricciones que deben gestionarse cuidadosamente para obtener un rendimiento óptimo y un uso eficiente de los recursos disponibles.

7. Trabajo en Equipo

7.1 Descripción del reparto de tareas

Los tres fuimos los responsables de implementar una arquitectura general del diseño del flujo de ETL utilizando el lenguaje de programación C# con integración de módulo de descomposición de datos y con la supervisión en un repositorio de GitHub, unificando todo esto en un documento final que los tres estuvimos desarrollando.

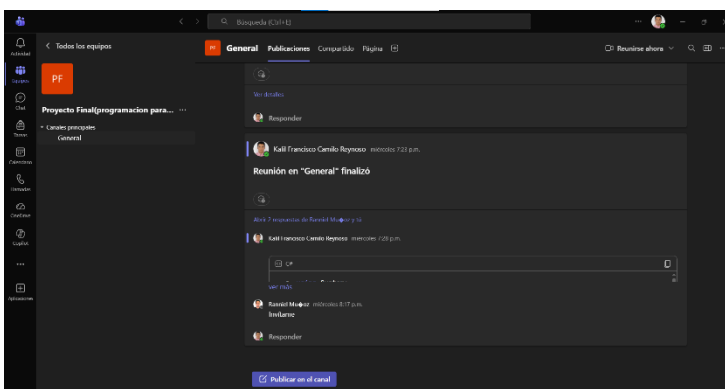
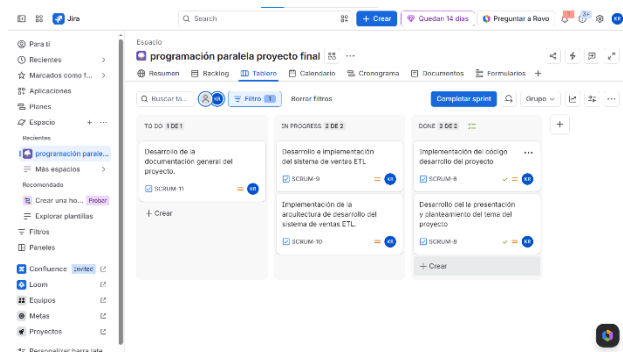
También a través de una llamada de Teams estuvimos diseñando los módulos de prueba de rendimiento, implementando ciertas métricas utilizando el stopwatch, calculando el speed up y la validación de las matrices de datos para la descomposición de estos datos.

Por último, nos estuvimos encargando de tener una buena estructuración para la simulación de transacciones que son masivas, en donde se estarían configurando el mecanismo de sincronización por bloques.

7.2 Herramientas utilizadas para coordinación

Las herramientas que estuvimos utilizando para el desarrollo de este proyecto se puede dividir en dos partes. La primera sería en el control de versiones, en donde estuvimos utilizando la herramienta de GitHub, en donde se estuvo subiendo constantemente commits y cambios de el proyecto, sea tanto a nivel de código como a nivel de documentación.

Para la comunicación entre los integrantes del grupo y la gestión de tareas estuvimos utilizando herramientas como WhatsApp y un equipo en Teams para las reuniones. Para la asignación de tareas estuvimos utilizando la plataforma de Jira.



Conclusión

Los principales aprendizajes técnicos que podemos obtener mediante este proyecto es que pudimos comprobar de una forma práctica la gran importancia que tiene la descomposición de datos para la transformación de problemas intensivos de procesamiento en carga distribuidas de forma eficiente.

También con el trabajo en equipo priorizamos el uso correcto de las herramientas de TPL para poder simplificar enormemente la gestión de los hilos en comparación con la manipulación manual de thread, reduciendo errores de concurrencia.

El principal reto para el desarrollo de este proyecto pudo haber sido la gestión de desarrollo y trabajo a nivel de las tareas que cada compañero tuvo que hacer, ya que tuvimos que distribuirlas para que todos prácticamente pudiéramos hacer lo mismo y de forma correcta.

Y también un reto a nivel de desarrollo técnico pudo haber sido la sincronización de los resultados de los resultados parciales de múltiples hilos sin generar condiciones de carrera o bloqueos lentos, en donde lo pudimos superar implementando una estrategia de acumulación basada en variables locales por tareas y agregación finalmente utilizando el Task.WhenAll, evitando bloqueos de forma innecesaria en secciones críticas.

También para **las posibles mejoras en el futuro** pudiéramos integrar procesamientos distribuidos real mediante múltiples nodos físicos, además de implementar una arquitectura master worker por red.

Implementar un apartado front end en donde también podríamos migrar las persistencias de los datos consolidados hacia un motor de base de datos relacionado y que sea bastante robustos como SQL Server, optimizando las conexiones concurrentes con connection pool.

REFERENCIAS

Microsoft Developer Network (MSDN). *Task Parallel Library (TPL) in .NET*. Recuperado de <https://learn.microsoft.com>.

Microsoft Azure Architecture Center (Extracción, Transformación y Carga - ETL por Lotes):

<https://learn.microsoft.com/es-es/azure/architecture/data-guide/relational-data/etl>

Microsoft Learn (.NET - Task Parallel Library / TPL):

<https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/task-parallel-library-tpl>

Anexos

<https://github.com/Kalil-Camilo-009/Proyecto-final-de-programaci-n-paralela>

<https://github.com/Kalil-Camilo-009/Proyecto-final-de-programaci-n-paralela/tree/main/metrics>

<https://github.com/Kalil-Camilo-009/Proyecto-final-de-programaci-n-paralela/tree/main/docs>